Universidad del Rosario
School of Science and Engineering
Operating Systems

Deliverable 1

Students:

- Carlos Arturo Galvis Mojica
- Shalem Shaged
- Jesus Esteban Peña Avila

1. Introduction

For this deliverable, three scheduling algorithms were developed and implemented for UR-OS, and their performance was subsequently compared. The algorithms implemented were:

- Shortest Job First Non-Preemptive (SJF-NP)
- Shortest Job First Preemptive (SJF-P / SRTF)
- Round Robin (RR)

In addition, several performance metrics were implemented to evaluate and contrast the algorithms, including CPU utilization, throughput, turnaround time, waiting time, response time, and number of context switches.
The three scheduling algorithms were integrated into the UR-OS architecture already in place, and each was tested against the simulation scenarios supplied by the simulator. The results generated by the simulator were then contrasted against both the expected values and the results obtained through manual calculation.

2. Scheduler Implementations

2.1 SJF Non-Preemptive

File: SJF_NP.java

```
package ur_os;

public class SJF_NP extends Scheduler{

    SJF_NP(OS os){
        super(os);
    }

    @Override
    public void getNext(boolean cpuEmpty) {

        if (!cpuEmpty || processes.isEmpty()) {
            return;
        }

        Process candidato = processes.get(0);
        for (Process p : processes) {
```

```
            if (p.getRemainingTimeInCurrentBurst() <
candidato.getRemainingTimeInCurrentBurst()) {
                candidato = p;
            } else if (p.getRemainingTimeInCurrentBurst() ==
candidato.getRemainingTimeInCurrentBurst() && p != candidato) {
                candidato = tieBreaker(candidato, p);
            }
        }

        processes.remove(candidato);
        os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, candidato);
    }

    @Override
    public void newProcess(boolean cpuEmpty) {} //Non-preemtive

    @Override
    public void IOReturningProcess(boolean cpuEmpty) {} //Non-preemtive

}
```

Description

The non-preemptive SJF algorithm chooses the process with the shortest remaining CPU burst from the ready queue.
The main logic is implemented in getNext(). The scheduler makes a selection only when the CPU is empty. This ensures non-preemptive behavior, since a process that is already running cannot be interrupted when a new process arrives.
The getRemainingTimeInCurrentBurst() method is used to compare the remaining CPU burst time of each process. When two processes have the same remaining time, tieBreaker() determines which one should run first.
Once the process with the shortest burst is selected, it is removed from the ready queue and transferred to the CPU through the SCHEDULER_RQ_TO_CPU interrupt.
The newProcess() and IOReturningProcess() methods are left empty because neither the arrival of a new process nor the return of a process from I/O should interrupt the process that is currently running on the CPU.

2.2 SJF Preemptive

File: SJF_P.java

```
package ur_os;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class SJF_P extends Scheduler{

    SJF_P(OS os){
        super(os);
    }
```

```
    @Override
    public void newProcess(boolean cpuEmpty){
        if (!cpuEmpty) {
            os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ, null);
            addContextSwitch();
        }
    }

    @Override
    public void IOReturningProcess(boolean cpuEmpty){
        if (!cpuEmpty) {
            os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ, null);
            addContextSwitch();
        }
    }

    @Override
    public void getNext(boolean cpuEmpty) {

        if (!cpuEmpty || processes.isEmpty()) {
            return;         }

        Process candidato = processes.get(0);
        for (Process p : processes) {
            if (p.getRemainingTimeInCurrentBurst() <
candidato.getRemainingTimeInCurrentBurst()) {
                candidato = p;
            } else if (p.getRemainingTimeInCurrentBurst() ==
candidato.getRemainingTimeInCurrentBurst() && p != candidato) {
                candidato = tieBreaker(candidato, p);
            }
        }

        processes.remove(candidato);
        os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU, candidato);
    }

}
```

Description

It uses the same shortest-job selection criterion as SJF-NP, it allows the process currently running on the CPU to be interrupted.
When a new process arrives or a process returns from I/O, the scheduler make a SCHEDULER_CPU_TO_RQ interrupt. This moves the currently running process back into the ready queue. After that, the scheduler evaluates all processes in the queue again and chooses the one with the shortest remaining CPU burst.
Since the scheduler can make a new decision even while another process is running, SJF-P can provide faster response times for shorter processes. However, this behavior may also result in more context switches compared to the non-preemptive version.

2.3 Round Robin

File: RoundRobin.java

```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
default.txt to change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
edit this template
 */
package ur_os;

/**
 *
 * @author prestamour
 */
public class RoundRobin extends Scheduler{

    int q;
    int cont;
    boolean multiqueue;

    RoundRobin(OS os){
        super(os);
        q = 5;
        cont=0;
    }

    RoundRobin(OS os, int q){
        this(os);
        this.q = q;
    }

    RoundRobin(OS os, int q, boolean multiqueue){
        this(os);
        this.q = q;
        this.multiqueue = multiqueue;
    }



    void resetCounter(){
        cont=0;
    }

    @Override
    public void getNext(boolean cpuEmpty) {

        if (cpuEmpty) {

            if (!processes.isEmpty()) {
                Process candidato = processes.poll();
                os.interrupt(InterruptType.SCHEDULER_RQ_TO_CPU,
candidato);
                resetCounter();
            }
            return;
        }
```

```
        cont++;

        if (cont == q) {

        if (!processes.isEmpty()) {

                Process siguiente = processes.poll();
                os.interrupt(InterruptType.SCHEDULER_CPU_TO_RQ,
siguiente);
                addContextSwitch();
            }

            resetCounter();
        }
    }


    @Override
    public void newProcess(boolean cpuEmpty) {} //Non-preemtive in this
event

    @Override
    public void IOReturningProcess(boolean cpuEmpty) {} //Non-preemtive
in this event

}
```

In this implementation, the time quantum is set to four simulation cycles.
The quantumCounter variable tracks how long the current process has been running. Once the counter reaches the defined quantum, the running process is removed from the CPU and placed at the end of the ready queue.
The scheduler then selects the next process following FIFO (First-In, First-Out) order.
Round Robin does not immediately interrupt the current process when a new process arrives or when a process returns from I/O. Instead, preemption mainly happens when the time quantum expires.
This scheduling method provides better fairness because every process gets regular access to the CPU. However, it generally causes more context switches compared to SJF.

3. Metrics

```
public double calcCPUUtilization() {

    if (clock == 0) return 0;

    int busyCycles = clock - cpucount; // cpucount cuenta los ciclos
con CPU vacía
    return ((double) busyCycles / clock);
}

public double calcTurnaroundTime() {

    double totalTurnaround = 0;
    int finishedCount = 0;
```

```
    for (Process p : processes) {
        if (p.getState() == ProcessState.FINISHED) {
            totalTurnaround += (p.getTime_finished() -
p.getTime_init());
            finishedCount++;
        }
    }

    if (finishedCount == 0) return 0;

    return totalTurnaround / finishedCount;
}

public double calcThroughput() {
    if (processes.isEmpty() || clock == 0) return 0;

    int finishedCount = 0;
    for (Process p : processes) {
        if (p.getState() == ProcessState.FINISHED) {
            finishedCount++;
        }
    }

    return (double) finishedCount / clock;
}

public double calcAvgWaitingTime() {

    double totalWaiting = 0;
    int finishedCount = 0;

    for (Process p : processes) {
        if (p.getState() == ProcessState.FINISHED) {
            int turnaround = p.getTime_finished() - p.getTime_init();

            totalWaiting += (turnaround - p.getBurstTime());
            finishedCount++;
        }
    }

    if (finishedCount == 0) return 0;

    return totalWaiting / finishedCount;
}

//Everytime a process is taken out from memory, when a interruption
occurs
public double calcAvgContextSwitches() { //solo gantt

    if (processes.isEmpty() || execution.isEmpty()) {
        return 0;
    }

    int switches = 0;
    int prevPID = -1;
    for (int actPID : execution) {
        if (actPID != prevPID) {
```

```
            if (actPID != -1) {
                switches++;
            }
            prevPID = actPID;
        }
    }

    return (double) switches / processes.size();
}


//Just context switches based on the execution timeline
public double calcAvgContextSwitches2() {

    if (processes.isEmpty() || execution.isEmpty()) {
        return 0;
    }

    int switches = 0;
    int prevPID = -1;

    // Cambios que aparecen directamente en el Gantt
    for (int actPID : execution) {
        if (actPID != prevPID) {
            if (actPID != -1) {
                switches++;
            }
            prevPID = actPID;
        }
    }


    int schedulerSwitches = os.getTotalContextSwitches();


    return (double) (switches + schedulerSwitches-1) /
processes.size();
}


public double calcResponseTime() {

    double total = 0;
    int startedCount = 0;

    for (Process p : processes) {
        int rt = p.getResponseTime();
        if (rt != -1) { // -1 significa que el proceso nunca llegó a
ejecutarse en CPU
            total += rt;
            startedCount++;
        }
    }

    if (startedCount == 0) return 0;
```

```
    return (total / startedCount)-1;
}
```

Description of the indicators

CPU Utilization measures the percentage of simulation time during which the CPU is executing a process.

CPU Utilization = Busy CPU cycles / Total cycles

Throughput represents the number of completed processes per simulation cycle.

Throughput = Completed processes / Total cycles

Turnaround Time measures the total time that a process spends in the system.

Turnaround Time = Finish Time - Arrival Time

Waiting Time represents the time during which the process is waiting instead of executing its programmed CPU or I/O bursts.

Waiting Time = Turnaround Time - Total Burst Time

Response Time represents the time between the arrival of a process and its first CPU execution.

Response Time = First Execution Time - Arrival Time

Two context-switch measurements are included. The first one analyzes visible PID transitions in the execution timeline. The second combines the transitions observed in the Gantt with context-switch events explicitly registered by the scheduler

4. Comparison of Results

The implementations were tested with the Simpler and Simpler2 scenarios included in UR-OS.
The simulator results were compared with the expected results provided with the project and with manual calculations.

4.1 Simpler 1 Scenario

Algorithm	Cycles	CPU Utilization	Throughput	Avg. Turnaround	Avg. Waiting	Context Gantt	Context Complete	Avg. Response
SJF-NP	43	0.9302	0.0930	22.75	9.00	2.00	2.00	6.25
SJF-P	43	0.9302	0.0930	22.75	9.00	2.25	2.75	5.50
RR	40	1.0000	0.1000	29.50	15.75	3.25	3.25	2.00

For this scenario, the simulator results matched the expected and manually calculated values.
SJF-NP and SJF-P achieved the same average waiting and turnaround times. However, SJF-P

reduced the average response time from 6.25 to 5.50 because it can respond more quickly when shorter processes arrive.
Round Robin achieved the best average response time, with 2.00, but it also resulted in higher average waiting and turnaround times, as well as a greater number of context switches.

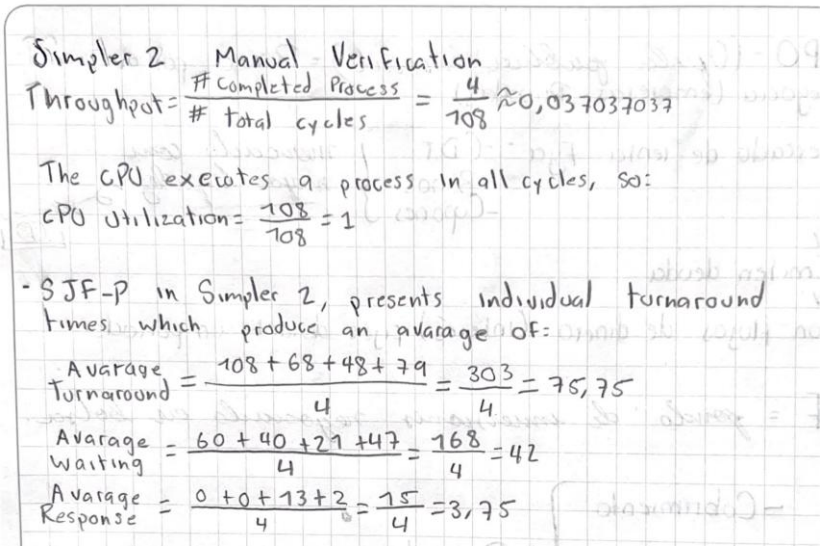
4.2 Simpler2 Scenario

Algorithm	Cycles	CPU Utilization	Throughput	Avg. Turnaround	Avg. Waiting	Context Gantt	Context Complete	Avg. Response
SJF-NP	108	1.0000	0.0370	76.75	43.00	2.00	2.00	13.50
SJF-P	108	1.0000	0.0370	75.75	42.00	2.25	3.50	3.75
RR	108	1.0000	0.0370	87.75	54.00	7.25	7.50	4.00

The second scenario makes the differences between the scheduling algorithms more noticeable.
All three schedulers achieved 100% CPU utilization and the same throughput because the CPU remained active throughout the simulation, and all four processes were completed within 108 cycles.
SJF-P achieved the best average turnaround and waiting times
The biggest difference between the two SJF approaches can be seen in their response times. This significant improvement occurs because the preemptive version can interrupt the process currently running and select a shorter job when it becomes available.
Round Robin achieved a response time of 4.00, which is also considerably better than SJF-NP. However, it had the highest average turnaround and waiting times
Round Robin also generated the most context switches because processes are repeatedly interrupted after reaching the four-cycle time quantum.

4.3 Manual Verification

The performance metrics were also manually checked against the process execution timelines.



These manual values align with the simulator output. Comparing the other tested algorithms similarly confirmed consistency across manual calculations, expected results, and the simulator.

5. Conclusion

The SJF Non-Preemptive, SJF Preemptive, and Round Robin algorithms were successfully implemented within the UR-OS simulator, along with the performance evaluation indicators required to assess them. Testing was carried out using the Simpler and Simpler2 scenarios, and in both cases the simulator's output matched the expected results as well as the values obtained through manual calculation, confirming the correct integration of the implementations into the existing architecture.

The results obtained are consistent with the theoretical behavior expected from each strategy. SJF Non-Preemptive generated the lowest scheduling overhead, since it lets a process finish its current burst before a new one is chosen, but this same characteristic led to weaker response times whenever short processes arrived while another one was already running. SJF Preemptive, on the other hand, was able to reconsider the shortest remaining burst as soon as new processes became available, which improved response, waiting, and turnaround times; this responsiveness came at the cost of a higher number of context switches. Round Robin, for its part, distributed CPU time fairly among processes through its fixed quantum and delivered good response times, but the repeated quantum expirations resulted in the highest context-switch overhead and in higher waiting and turnaround times overall.

Taken together, these findings indicate that no single scheduler outperforms the others across every metric; instead, each algorithm reflects a different trade-off between efficiency, responsiveness, fairness, and scheduling overhead. This confirms that the simulations behave in line with the theoretical foundations of the three scheduling algorithms and validates the correct integration of both the schedulers and the performance indicators into UR-OS.